

```
manchodu@srini:~/OSLAB$ nano 41.c
manchodu@srini:~/OSLAB$ gcc 41.c -o 41.c
gcc: fatal error: input file '41.c' is the same as output file
compilation terminated.
manchodu@srini:~/OSLAB$ gcc 41.c -o 41
manchodu@srini:~/OSLAB$ ./41
```

```
=====
SIGNAL HANDLER MANAGEMENT
=====
```

Signal handlers registered successfully.

Process PID: 2129

Configured Signals:

SIGINT -> Ctrl+C

SIGTERM -> Termination request

SIGUSR1 -> User-defined signal 1

SIGUSR2 -> User-defined signal 2

Testing signal events...

Send signals from another terminal using:

kill -SIGUSR1 2129

kill -SIGUSR2 2129

kill -SIGTERM 2129

Press Ctrl+C to test SIGINT.

^C

```

manchodu@Srini:~/OSLAB$ cat 41.c
/* signal_handler_management.c
 *
 * Register Handlers
 * Configure Signal Actions
 * Handle Interrupts
 * Manage Signal Context
 * Verify Stability
 * Test Signal Events
 *
 * Compile:
 * gcc signal_handler_management.c -o signal_handler_management
 */

#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>
#include <signal.h>
#include <string.h>

volatile sig_atomic_t signal_received = 0;

void signal_handler(int sig, siginfo_t *info, void *context)
{
    (void)context;

    signal_received = sig;

    printf("\n=====\\n");
    printf("          SIGNAL EVENT\\n");
    printf("=====\\n");

    printf("Signal Number : %d\\n", sig);
    printf("Sender PID    : %d\\n", info->si_pid);

    if (sig == SIGINT)
        printf("Signal Type   : SIGINT (Ctrl+C)\\n");
    else if (sig == SIGTERM)
        printf("Signal Type   : SIGTERM\\n");
    else if (sig == SIGUSR1)
        printf("Signal Type   : SIGUSR1\\n");
    else if (sig == SIGUSR2)
        printf("Signal Type   : SIGUSR2\\n");

    printf("Handler executed successfully.\\n");
    printf("=====\\n");
}

```

```

    sigemptyset(&action.sa_mask);

    action.sa_flags = SA_SIGINFO | SA_RESTART;

    if (sigaction(signal_number, &action, NULL) == -1)
    {
        perror("sigaction");
        exit(EXIT_FAILURE);
    }
}

int main()
{
    printf("=====\n");
    printf("      SIGNAL HANDLER MANAGEMENT\n");
    printf("=====\n");

    configure_signal(SIGINT);
    configure_signal(SIGTERM);
    configure_signal(SIGUSR1);
    configure_signal(SIGUSR2);

    printf("\nSignal handlers registered successfully.\n");

    printf("\nProcess PID: %d\n", getpid());

    printf("\nConfigured Signals:\n");
    printf("SIGINT  -> Ctrl+C\n");
    printf("SIGTERM -> Termination request\n");
    printf("SIGUSR1 -> User-defined signal 1\n");
    printf("SIGUSR2 -> User-defined signal 2\n");

    printf("\nTesting signal events...\n");

    printf("\nSend signals from another terminal using:\n");
    printf("kill -SIGUSR1 %d\n", getpid());
    printf("kill -SIGUSR2 %d\n", getpid());
    printf("kill -SIGTERM %d\n", getpid());

    printf("\nPress Ctrl+C to test SIGINT.\n");

    while (1)
    {
        pause();

        if (signal_received == SIGTERM)
        {

```

```
printf("\nsend signals from another terminal using:\n");
printf("kill -SIGUSR1 %d\n", getpid());
printf("kill -SIGUSR2 %d\n", getpid());
printf("kill -SIGTERM %d\n", getpid());

printf("\nPress Ctrl+C to test SIGINT.\n");

while (1)
{
    pause();

    if (signal_received == SIGTERM)
    {
        printf("\nSIGTERM received. Exiting safely.\n");
        break;
    }

    signal_received = 0;
}

printf("\nSignal handling test completed successfully.\n");

return 0;
}
manchodu@Srini:~/OSLAB$ |
```

```
manchodu@Srini:~/OSLAB$ nano m42.c
manchodu@Srini:~/OSLAB$ gcc m42.c -o m42
manchodu@Srini:~/OSLAB$ ./m42
=====
          FOREGROUND SIGNAL HANDLING SHELL
=====
Shell PID   : 2182
Shell PGID  : 2182

Shell signal handlers configured.

=====
1. Start Foreground Job
2. Show Shell Information
3. Exit
=====
Enter choice: 1

Shell: Foreground job PID=2185 PGID=2185

-----
Foreground Job Started
PID   : 2185
PGID  : 2185
-----

Job is running...
Press Ctrl+C to send SIGINT.
Foreground job PID 2185 is working...
^C
Shell: Forwarding SIGINT to foreground process group 2185

Job terminated by signal 2.

Shell: Foreground job state updated.

=====
1. Start Foreground Job
2. Show Shell Information
3. Exit
=====
Enter choice: 3

Shell exiting safely.
```

```
manchodu@Srini:~/OSLAB$ cat m42.c
/* foreground_interrupt_handling.c
 *
 * Capture SIGINT
 * Forward Signals
 * Protect Shell Process
 * Terminate Foreground Jobs
 * Update Job States
 * Test Interrupt Handling
 *
 * Compile:
 * gcc foreground_interrupt_handling.c -o foreground_interrupt_handling
 */

#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>
#include <signal.h>
#include <sys/types.h>
#include <sys/wait.h>
#include <errno.h>

pid_t foreground_pgid = -1;

void shell_sigint_handler(int sig)
{
    if (foreground_pgid > 0)
    {
        printf("\nShell: Forwarding SIGINT to foreground process group %d\n",
            foreground_pgid);

        kill(-foreground_pgid, sig);
    }
    else
    {
        printf("\nShell: SIGINT received, but no foreground job is running.\n");
    }
}

void shell_sigquit_handler(int sig)
{
    if (foreground_pgid > 0)
    {
        printf("\nShell: Forwarding SIGQUIT to foreground process group %d\n",
            foreground_pgid);

        kill(-foreground_pgid, sig);
    }
}

void install_shell_handlers()
{
    struct sigaction action;
```

```

}

void install_shell_handlers()
{
    struct sigaction action;

    action.sa_handler = shell_sigint_handler;
    sigemptyset(&action.sa_mask);
    action.sa_flags = SA_RESTART;

    if (sigaction(SIGINT, &action, NULL) == -1)
    {
        perror("sigaction SIGINT");
        exit(EXIT_FAILURE);
    }

    action.sa_handler = shell_sigquit_handler;

    if (sigaction(SIGQUIT, &action, NULL) == -1)
    {
        perror("sigaction SIGQUIT");
        exit(EXIT_FAILURE);
    }
}

void protect_shell_process()
{
    signal(SIGTSTP, SIG_IGN);
    signal(SIGTTIN, SIG_IGN);
    signal(SIGTTOU, SIG_IGN);
}

void terminate_foreground_job(pid_t pgid)
{
    if (pgid > 0)
    {
        printf(
            "\nTerminating foreground job PGID: %d\n",
            pgid
        );

        if (kill(-pgid, SIGTERM) == -1)
        {
            perror("kill SIGTERM");
        }
    }
}

void run_foreground_job()
{
    pid_t pid;
    int status;

    pid = fork();

    if (pid < 0)

```

```

        perror("fork");
        return;
    }

    if (pid == 0)
    {
        /*
         * Create a new process group.
         */
        setpgid(0, 0);

        /*
         * Restore default signal behavior for the child.
         */
        signal(SIGINT, SIG_DFL);
        signal(SIGQUIT, SIG_DFL);
        signal(SIGTSTP, SIG_DFL);

        printf("\n-----\n");
        printf("Foreground Job Started\n");
        printf("PID   : %d\n", getpid());
        printf("PGID  : %d\n", getpgrp());
        printf("-----\n");

        printf("\nJob is running...\n");
        printf("Press Ctrl+C to send SIGINT.\n");

        while (1)
        {
            printf("Foreground job PID %d is working...\n",
                    getpid());

            sleep(2);
        }

        exit(0);
    }

    /*
     * Parent creates process group for child.
     */
    setpgid(pid, pid);

    foreground_pgid = pid;

    printf(
        "\nShell: Foreground job PID=%d PGID=%d\n",
        pid,
        foreground_pgid
    );

    /*
     * Wait for foreground job.
     */
    while (1)
    {

```

```

        &status,
        WUNTRACED
    );

    if (result == -1)
    {
        if (errno == EINTR)
            continue;

        perror("waitpid");
        break;
    }

    if (WIFEXITED(status))
    {
        printf(
            "\nJob exited normally with status %d.\n",
            WEXITSTATUS(status)
        );

        break;
    }

    if (WIFSIGNALED(status))
    {
        printf(
            "\nJob terminated by signal %d.\n",
            WTERMSIG(status)
        );

        break;
    }

    if (WIFSTOPPED(status))
    {
        printf(
            "\nJob stopped by signal %d.\n",
            WSTOPSIG(status)
        );

        break;
    }
}

foreground_pgid = -1;

printf("\nShell: Foreground job state updated.\n");
}

int main()
{
    int choice;

    printf("=====\n");
    printf("      FOREGROUND SIGNAL HANDLING SHELL\n");

```

```

while (1)
{
    printf("\n=====\\n");
    printf("1. Start Foreground Job\\n");
    printf("2. Show Shell Information\\n");
    printf("3. Exit\\n");
    printf("=====\\n");

    printf("Enter choice: ");
    scanf("%d", &choice);

    switch (choice)
    {
        case 1:
            run_foreground_job();
            break;

        case 2:
            printf("\\nShell PID : %d\\n", getpid());
            printf("Shell PGID : %d\\n", getpgrp());

            if (foreground_pgid > 0)
                printf(
                    "Foreground PGID : %d\\n",
                    foreground_pgid
                );
            else
                printf(
                    "Foreground PGID : None\\n"
                );

            break;

        case 3:
            if (foreground_pgid > 0)
            {
                terminate_foreground_job(
                    foreground_pgid
                );
            }

            printf("\\nShell exiting safely.\\n");
            return 0;

        default:
            printf("Invalid choice.\\n");
    }
}

return 0;
}

```